# InpaintAR

## A Comparison of different InPainting Approaches for Real-Time Use

Sebastian Gradwohl


BACHELOR THESIS

submitted to the
Bachelor's Degree Program
Software Engineering

at the
University of Applied Sciences Upper Austria
in Hagenberg

2026

Supervisor: FH-Prof. Dr. Christoph Anthes, MSc

# Declaration

I hereby declare and confirm that this thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I further declare that this or similar work has not been submitted for credit elsewhere. This printed copy is identical to the submitted electronic version.

Hagenberg, February 1, 2026

Sebastian Gradwohl

# Contents

# Abstract

Due to technological advances in both Hardware and Software-Integration, Augmented Reality is getting more popular with each year.
One of the issues still prevalent in Augmented Reality Software is the distraction caused by clutter surrounding the digital content in use cases like immersive analytics.

This bachelors thesis focuses on the analysis of the different methods and algorithms for the concealment of the clutter to provide a clean and non-distracting environment for the digital object to be displayed on.
To accomplish this, a tool to select planes on which objects are detected to subsequently be hidden through inpainting will be implemented. With this tool, several methods and algorithms will be evaluated in terms of both quality and performance to provide believable and uncluttered environments, whilst at the same time minimizing the use of computational resources and time.
These findings are compared to each other so readers can get an overview on both the advantages and disadvantages each of the analyzed approaches delivers.

# Kurzfassung

Aufgrund der technologischen Fortschritte, sowohl in der Hardware, als auch in der Software-Integration, steigt die Beliebtheit von Augmented Reality mit jedem Jahr an. Ein sehr gängiges Problem in aktueller Augmented Reality Software ist die Ablenkung, welche durch physische Objekte, die um den digitalen Inhalten liegen. Beispielsweise bei der immersiven Analyse von Daten.

In dieser Bachelorarbeit liegt der Fokus auf der Analyse verschiedener Methoden und Algorithmen zur Verdeckung dieser Objekte, wodurch eine aufgeräumte, saubere Umgebung für die digitalen Elemente geschaffen wird.
Um dies zu erreichen wird ein Werkzeug entwickelt, welches erlaubt Flächen zu wählen, welche dann dazu verwendet werden, um Objekte auf diesen Flächen zu finden und darauffolgend durch die Nutzung von Inpainting zu verdecken. Mithilfe dieses Werkzeugs werden mehrere Methoden und Algorithmen sowohl auf Qualität, als auch auf Leistung evaluiert, um sowohl die Glaubbarkeit und Ordnung der generierten Umgebungen, während parallel dazu ein Fokus auf die Minimierung von Rechenresourcen und Laufzeit gelegt wird.
Diese Ergebnisse werden zueinander verglichen, sodass Leser sowohl über die Vor- als auch Nachteile der jeweiligen Ansätze einen Überblick bekommen.

# Chapter 1

# Introduction

The domain of Augmented Reality (AR) spans all kinds of real-time visualization approaches wherein an otherwise real environment is augmented with virtual objects [Mil+95] While many still considerAR a nieche, the user-numbers tell a different story. In 2023 Statista reported about 983 Million users to be using AR on their mobile devices. [Sta24].

One of the issues still common in AR Applications is the overload of information. Whenever users are supposed to focus on the virtual objects at hand, the visibility of physical objects may serve as a distraction. (cf. Cheng et. al. [Che+22]) An existing solution is the use of Diminished Reality (DR). This concept can be interpreted as a specific use case of AR, which aims to hide or remove certain physical objects from the viewport in real time.[MF02] (see 1.1). In the case of the tool presented in this thesis, DR can be used to reduce the visual clutter.

To achieve this the visual clutter from the viewport is intentionally drawn over, artificially damaging the frame shown to the user on the display. To fix such damaged images, image inpainting is one of the techniques available and the one used in this thesis.


**Figure 1.1: TODO SG; Add after first example can be done using the tool;** Real and Diminished scene. This example serves as an example of how DR may be used to reduce visual clutter. (a) shows the regular environment, (b) shows the same image with the visual clutter overdrawn, whilst (c) shows the cleaned up DR image, achieved through inpainting

## 1.1 Motivation

Clutter is an ever present phenomenon in our daily lives. The description of what exactly counts as clutter in terms of features, attributes or how to identify it objectively is still difficult to define [RLN07]. It has been proven through multiple experiments and studies that reducing visual distractions such as clutter improves a persons ability to focus and keep a longer attention span, see for example [MA23]

Distraction cannot only be caused by visual input, but also through sound. During the last few years the technology of noise cancellation has matured enough to allow the mass production of high-quality headphones with acoustic noise cancellation, therefore overcoming this negative effect. At the current moment there is very little awareness about the visual noise though. During an experiment done on which visual factors might impact focus, next to lighting or motions, information overload or distractions caused by objects such as food, animals, large textures or bright spots have been pointed out by the participants [Hon+24].

While there are existing solutions for achieving DR using inpainting, as seen for example in "DeepDR", a project to provide structure-aware RGB-D inpainting for DR [Gsa+24], the evaluation of which of the existing inpainting methods for not only this use case, but inpainting in a 3-dimensional real-time environment has largely been disregarded as of now.

Most publications put the focus on different components of the system like the detection of the objects, depth detection, user studies or optimizations of smaller visual artifacts. At most some evaluation is done when presenting new state of the art algorithms and solutions for inpainting, but a large overview of existing algorithms and solutions and the evaluation in a real-time setting has been missing as of now. Over the last years there have been numerous achievements in the space of inpainting methods. Ranging from classic, low-cost algorithms like the fast marching algorithm [SHD21] or the usage of matching textures using gaussian weighting to fill the affected regions [DRR19] to more modern machine-learning assisted high fidelity approaches like the DeepFill Deep-Learning model [CJL23].

The main goal of this analysis is to provide an easy to reference comparison of the different methodologies to be compared in terms of both performance and quality. Through this analysis, developers can make an educated choice on which methodology to use for their use case, be it high fidelity in use cases like PC-powered AR systems or low energy, low performance systems, more suited for mobile devices, be they mobile phones or standalone Mixed-Reality Head-Mounted Displays (HMDs). In addition, the tool built for this evaluation should be easy to extend for newer inpainting methods whilst staying easy to integrate into existing applications to allow for a quick and efficient tool to provide the clutter removal functionality to other AR projects.

## 1.2 Overview

This work includes an introduction to the underlying concepts and analyzed inpainting methods, followed by the conceptualization and implementation of the tool as well as the performance and quality evaluation for each of the analyzed inpainting methods. These topics are elaborated upon in the following five chapters:

- **Chapter 2: Fundamentals and Related Work**
  At the start of the thesis, this chapter serves to provide an introduction into visual clutter and why it matters to the domain of AR. The term of DR is explored into further detail, as well as the underlying logic of each of the analyzed inpainting methods and how inpainting may be accomplished for DR.
- **Chapter 3: Concept**
  The concept chapter goes into further detail on the architecture and design chosen for this tool. Next to explaining the different requirements and code libraries needed for the usage of the tool, some domains and usecases in which the tool might be used are also elaborated upon.
- **Chapter 4: Implementation**
  In this chapter each of the classes contained in this tool is explained briefly. In addition a small guide is being provided, as to how the tool could be implemented for the usage in existing projects. Some of the tools and principles used during the implementation are going to be elaborated on as well.
- **Chapter 5: Evaluation**
  After providing an overview of the testing setup and how visual clutter and performance are measured, the test scenarios will be presented, followed by the results for each of the analyzed methods.
- **Chapter 6: Conclusion and Future Work**
  At the end of the thesis all the results and the everything mentioned in the previous chapters is briefly summed up. Afterwards further ideas on how to expand and improve the tool in the future are provided to improve both the developer and user experience even further.

# Chapter 2

# Fundamentals and Related Work

Before starting with the introduction to the InpaintAR tool some general knowledge and information needs to be further elaborated upon. All of the concepts relevant for the tool, and which algorithms and logics it uses to achieve its goal of reducing the users distraction are being presented, starting from a more detailed explanation on Visual Clutter and why it is relevant in MR, to a more detailed introduction to DR, inpainting, how those two concepts can be combined and the inpainting methods analyzed in the tool and this thesis.

## 2.1 What is Visual Clutter

Clutter can be seen as a state in which items, their placement or some features causes performance degradations in certain tasks. It can lead to a decrease in object recognition performance or cause issues when searching for certain objects. In addition clutter also seems to have a detrimental effect to short-term memory, whereas not only the number of elements, but their features can have a negative effect aswell [RLN07].

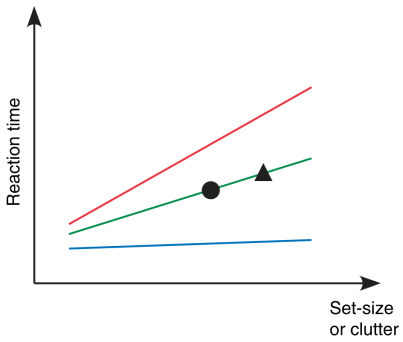



**Figure 2.1:** Cited from [RLN07], Page 3; It shows a comparison between reaction time and clutter to predict performance. The difficulty of the given task may also change the increase in reaction time compared to the clutter, therefore 3 curves are provided instead of one.

### 2.1.1 Why does it matter for Mixed Reality

Mixed Reality (MR), as defined by Paul Milgram in 1994, can be seen as an environment in which real and virtual objects are presented together through a single display. Therefore AR, DR and Virtual Reality (VR) can all be seen as subcategories of MR, which are placed somewhere in the virtuality continuum which spans a completely real environment to a completely virtual one [MK94].

As defined by S. Mann and J.Fung back in 2002, AR aims to enhance reality through the addition of virtual objects, whilst DR aims to remove physical objects from the users perception [MF02].

Whilst during the design of traditional software and information visualization, information overload and clear user interfaces need to be carefully considered. In these domains all of the factors which may induce clutter and information overload can be controlled internally in the visualization or software [RLN07].

AR is uniquely flawed in that regard due to it enhancing the surrounding environment of the user. This in turn means there could be a lot of external factors which may cause clutter, starting from the pure concept and interactivity of AR oftentimes allowing objects to be placed freely in the environment and therefore cluttered environments to the space constraints of the user themselves.

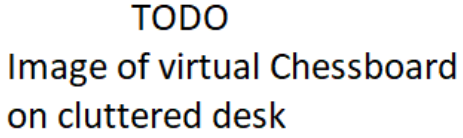



**Figure 2.2: TODO SG; Add after UseCase Example done;** An example of virtual objects enhancing an already cluttered environment and therefore worsening the existing visual clutter

## 2.2 Diminished Reality

DR was first introduced as a subsection of MR by Mann [MF02], often also called Mediated Reality, due to the concept of software mediating both the physical and virtual objects the user gets to see. DR aims to remove or diminish real world objects from the perception of the user by hiding them from their view. This can range from completely hiding the objects or making them slightly transparent, so the user can still recognize the object when looking directly at it, but it is less perceived when looking at different objects [Che+22].

Many experiments have been done using DR already. Ranging from visually removing the front vehicle, which is being overtaken from a drivers viewport for safety improvements during overtaking [Ram+16] to interior design previews and furniture placement previews, as seen in [Sil17].

These works use similar concepts to the one implemented in the tool built for this thesis, working with inpainting algorithms to hide physical objects for the users benefit, be it decision making for interior designs, increasing driver safety during overtaking, or in the case of this thesis, in reducing distractions caused by clutter.

There have also been user studies to experiment and find out which effects are the most comfortable and effective for users, ranging from simply reducing the opacity of objects, to outlining, blurring or desaturating the unwanted objects. According to the experiments done in [Che+22] opacity reduction and outlining provides the best results. This can be interpreted to mean, the less a user notices objects the less they are distracted by them. In addition another experiment measured how users react to being able to decide which objects should be hidden compared to every object being detected and hidden automatically. The findings of this experiment have shown a preference for the users to decide for themselves, which objects should be hidden and which should stay [Che+22].

These findings were directly involved in the conceptualization of the InpaintAR tool, as to how objects should be hidden and allowing for the user to select the areas and therefore the objects they want to hide for themselves instead of the software deciding.

## 2.3 Inpainting

Whilst DR itself is a great concept for the usecase of reducing clutter, the problem remains of what to overlay over the objects to hide them. A single color block would prove for the easiest solution, but it would ruin the users immersion and could prove to be even more distracting than the objects which were in the covered area beforehand. Therefore this overlay needs to be filled with a plausible background. Inpainting provides a good solution to fill these regions of the viewport and create believable overlays.

### 2.3.1 What is inpainting

Image inpainting is based on the concept of recovering and completing missing regions in an image or removing objects in it. This concept has been around for many years, having been used in image restoration or removing damage caused by deterioration. In recent years it has gained more popularity due to the improvement of image processing tools and the advances of digital image editing. To achieve this several methods have been developed to achieve this effect automatically. These range from static sequential algorithms over CNN-based approaches, using neural networks with automatic deep learning up to GAN-based solutions using Generative adversarial networks for training the images of specialized inpainting models [Elh+20].

These categories and many of the provided methods provide adequate results for the inpainting of images, DR requires these methods to perform the inpainting as quickly as possible to adjust to the users movement and provide believable results. Therefore there are several methods specialized specific use of real-time inpainting. Some examples are the usage of edge-computing to outsource the inpainting work to a separate server, as seen in [Zha+22] or to improve the inpainting process through concepts such as feature reuse and mask propagation, as seen in [Liu+25].


**Figure 2.3:** An example of inpainting, (a) original image, (b) image with synthetic damage, (c) image fixed with texture synthesis of surrounding area pixels

### 2.3.2 Methods analyzed in this thesis

As there are many different methods currently available regarding inpainting, analyzing all of them would require a lot of time and be outdated quickly. Therefore, only a few candidates for each of the larger categories of inpainting methodologies are being looked at in detail to provide an overview.

From gaining an understanding, which categories correlate to certain desired benefits, readers may get a rough estimation of how other methodologies from the same category might perform. Of these methodologies, the following have been analyzed:

Many of the current inpainting methods are made to use trained models, which are fed large amounts of example data to receive good results. Since this does not seem very practical in practice, when trying to implement inpainting as a part of a larger system or project, the focus has therefore been put on regular algorithms and self-training models.

#### Fast Marching Method

Having first been proposed back in 1999 by Sethian, see [Set99], the Fast Marching Method (FMM) is an analytical method, which was first developed to solve boundary problems.

FMM has been proven by Telea to also be applicable to pixel-based problems, like inpainting. The algorithm works by creating an image smoothness estimator, which estimates the image smoothness of a given neighborhood to fill. These are used as a level set for the FMM algorithm to propagate the image information [Tel04].

The advantages of the algorithm are both its easy-to-implement nature, allowing for an easy understanding of the algorithm, as well as its efficiency and quality considering the simplicity.

One of the disadvantages, which is present for most algorithmic methodologies, is the issue of larger missing regions. There have been proposals and solutions for this disadvantage for the FMM method though. Nurmala, Horikawa and Du proposed a solution using segmentation of the large missing region through structure propagation to split the missing region into multiple smaller missing image regions [SHD21]. This

makes the algorithm very attractive for real time DR use cases, as it is both performant and can handle missing regions of variable sizes

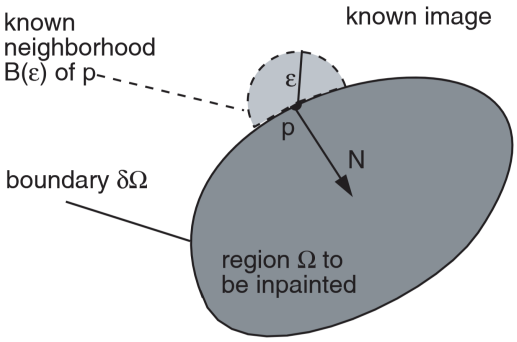



**Figure 2.4:** Cited from [Tel04]; Concept of the Fast-Marching Inpainting

The mathematical formula for the calculation of a single pixels value can be boiled down to this:

$$I(p) = \frac{\sum_{q \in B\epsilon(p)} w(p,q)[I(q) + \Delta I(q)(p-q)]}{\sum_{q \in B\epsilon(p)} w(p,q)}$$

$B\epsilon$ in this case describes the known area around the pixel. These points are all summed up and weighted by a normalizing function $w$. $\Delta I$ is the image gradient, which is estimated by the central differences of the neighboring pixels.

The weight function can be described as such

$$w(p,q) = dir(p,q) \times dst(p,q) \times lev(p,q)$$

whereas these three factors have the following formulas:

$$dir(p,q) = \frac{p-q}{||p-q||} \times N(p)$$

$$dst(p,q) = \frac{d_0^2}{||p-q||^2}$$

$$lev(p,q) = \frac{T_0}{1 + |T(p) - T(q)|}$$

$dir(p,q)$ in this case calculates the priority of the analyzed pixel $q$ in correlation to the pixel $p$s normal direction $N = \Delta T$. $dst(p,q)$ defines the distance between the known pixel $q$ and the yet unknown pixel $p$. Closer pixels should be weighted stronger than pixels which are further away. $lev(p,q)$ lastly is the level set distance of the two pixels, ensuring that pixels closer to the contour of $p$ contribute more to the resulting value of $p$ than those further away.
$d_0 and T_0$ reference the relative interpixel distance, usually set to 1. It was based on the

model of manual inpainting heuristics first introduced by Bertalimo, see [Ber+00], on how to paint a point by introducing colors from a small region around it [Tel04].

This algorithm itself can only calculate a value for one color channel. By applying it to all 3 channels of the RGB spectrum the algorithm can be used to calculate pixels for colored video frames.

### Nearest Neighbor Pixel Filling

### Nonlocal Texture Matching and Nonlinear Filtering

Proposed back in 2019, see [DRR19], this method aims to find existing neighboring areas, which match the existing surrounding texture to find a pixels value. Whilst this approach is a lot more complex compared to the rather simple FMM, it promises higher quality results, due to working through batches of pixels and trying to keep a consistent pattern within it.

It accomplishes this by first prioritizing the patches to be filled. This patch contains both a source region and the unknown part, in which the contour of the missing region is in the center of the patch.

The priority function is defined as

$$P(\Psi_p) = C(\Psi_p) \times D(\Psi_p), p \in \delta\Omega$$

In this function, $\Psi_p$ is defined as the selected patch, whereas the pixel p is defined as the center point of the patch, which is a part of the unknown area boundaries $\delta\Omega$. $C(\Psi_p)$ defines the confidence term, which is defined as the ratio of known pixels within the given patch. $D(\Psi_p)$ is defined as the dot product of the isophote vector of p ($\Delta I_p$). This vector is defined to go along the direction of equal pixel intensity lines of the known area.

$$D(\Psi_p) = \frac{\Delta I_p \times n_p}{I_{max}}$$

$$\Delta I_p = \frac{1}{2I_{max}}\{[I(x_p, y_p - 1) - I(x_p, y_p + 1)] \times i + [I(x_p + 1, y_p) - I(x_p - 1, y_p)] \times j\}$$

$$n_p = [M(x_p + 1, y_p) - M(x_p - 1, y_p)] \times i + [M(x_p, y_p + 1) - M(x_p, y_p - 1)] \times j$$

$n_p$ in this case defines the normal vector at the center pixel $p$. $M$ returns 1, in case the given coordinate in the parameters is inside of the known pixels, and 0 in case it is at the position of an unknown pixel. Lastly, i and j are defined as unit vectors in the x and y direction.

After the prioritized target patch is chosen, a nonlocal texture similarity (NLTS) measure is used to select canidate patches as sources for filling the target region. This NLTS is defined as such:

$$NLTS(I_p, I_q) = \frac{-||(I_p^2 - I_q^2)oG_p||1}{h^2}$$

$o$ is in this case defined at the element-wise product operator, whilst h is a user-selected parameter, which should be proportionately larger than the maximum color value (usually 255). $G_p$ is the Gaussian kernel over the target patch with p at its center.

Using this NLTS measure, a number of source regions may be collected. The more canidates the following algorithm has, the less error prone the inpainting.

After deciding on which canidate patches to pick, for each of the missing pixels, the mean value of all the canidates corresponding pixels is calculated and applied to the target pixel.
To support color, the isophote vector, used for the priority calculation, is computed as the average of all three color channels. The NLTS formular too is changed slightly, to sum up the value above for each of the three color channels [DRR19].

#### Exemplar-Based Image Inpainting with modified priorities

This method was first proposed 2015 by Deng, see [DHZ15].
For this method, many approaches are similar to the nonlocal texture matching, see 2.3.2. It was chosen as an additional method, as it was shown to be more performant in the benchmarks in [DRR19], although the results were apparently slightly worse. Therefore these two algorithms are used as a comparison of such exemplar-based inpainting algorithms compared to other methodologies.

Compared to the approach of the prior chapter, the priority function is overhauled to the following:

$$P(p) = \begin{cases} \mathrm{D(p)}, & \text{first phase} \\ \mathrm{C(p)}, & \text{second phase} \end{cases}$$

As for how many steps each phase gets, the first phase needs to be estimated or set statically. Afterwards the second phase may go on until the missing region is filled out completely.
To reduce the computational load of the similarity between two patches, the formula was changed accordingly:

$$\Psi_q = argmin\ d(\Psi_p, \Psi_q)$$

$d(\Psi_p, \Psi_q)$ in this case describes the sum of squared differences of the known pixels between the two patches.

After finding the patch with the highest similarity, this patch is used to fill the missing pixels of the target patch. This is one of the methodologies in which it differs to the approach of 2.3.2. Instead of picking multiple source patches and using mean values it only considers one [DHZ15].

This should therefore reduce the computational complexity, reducing the runtime of the process. In return this will causes the method to be a bit more error prone and for the output to potentially be of lower quality.

#### Regression

### 2.3.3 Inpainting in Diminished Reality

# Chapter 3

# Concept

Go into Detail about the overview, how it is built architecturally, what are the components, explained in further sections

## 3.1 Components

## 3.2 Requirements

Small explanation of what requirements exist, unity

### 3.2.1 Hardware

go into detail about meta quest 3

### 3.2.2 Meta AR Library

Explain the AR Library, how it is used / why needed

### 3.2.3 Unity XR Hand-Tracking Library

Explain Gestures / Hand Pose Recognition

## 3.3 Usecases

Go into Detail about how the tool might prove useful

### 3.3.1 Interactive Analysis

Maybe ask canthes if i can reference some of the interactive analysis projects from HIVE as references / images

### 3.3.2 AR Training Simulations

### 3.3.3 Entertainment Purposes

TODO add Image with virtual Chessboard on cluttered desk

# Chapter 4

# Implementation

Explain how it is implemented

## 4.1 Simulating the Headset for development

Explain how the Meta XR Simulator was used during implementation to allow for faster development without needing the headset as much

## 4.2 Area Selection

Go into Detail on how Hand Gesture Rectangle is evaluated to area and added as mask on top, how the mask is gotten from the area through snakes

## 4.3 Inpainting

Further Details about the Inpainting Algorithms, per Algorithm

## 4.4 Mask Overlay

How the inpainted content is put in front of the HMD Camera feed without getting into the way of AR elements

# Chapter 5

# Evaluation

When the Evaluation was conducted, in what environment

## 5.1 Testing Setup

### 5.1.1 Hardware Setup

1x Meta Quest 3 Standalone, 1x Desktop VR with dedicated GPU through Meta Quest 3 with USB-C

Maybe show headset setup, probably Ladder with Headset strapped to something so it can be tested without user input / movement

### 5.1.2 Measuring Quality for Inpainting

#### Quality Benchmarks

Use InPainting Quality Benchmarking, Part with Quality Benchmarking without knowing what it should look like, also mention that as a small issue in the evaluation

#### How visual Clutter is measured

See Paper Measuring Visual Clutter

### 5.1.3 How Performance is measured

Use Clock/Timer for the Inpainting Method from Call to Return + FrameTime + FrameRate Maybe use the Unity3D Profiler

## 5.2 Test Scenarios

### 5.2.1 Table with Pen and Notebook

as said in subsection + Visual Clutter Metric

### 5.2.2 Table with reflective see-through objects

Table with multiple glasses of water + Visual Clutter Metric

### 5.2.3 Heavily Cluttered Desk

Desk full with different kinds of junk, clean up all elements on top + Visual Clutter Metric

### 5.2.4 Large Objects

Probably entire Table / Desk + Visual Clutter Metric

## 5.3 Results

### 5.3.1 Quality Metrics

SubSubsection for each of the methods evaluated Graph of the quality metrics (Quality + Visual Clutter remaining) per test-scenario for each of the methods + explanation

### 5.3.2 Performance Metrics

SubSubsection for each of the methods evaluated Graph of the performance metrics per test-scenario for each of the methods + explanation

# Chapter 6

# Conclusion and Future Work

Still TODO

## 6.1 Future Work

### 6.1.1 Application to selective planes in VR environments

### 6.1.2 Combination with Dynamic Object Detection

SimpSON Paper, Single-Click-Distraction-Detection; Maybe mention RoboDK AI Model for Detecting Objects by Name and providing the user a table to select which objects should be hidden and shown Maybe adding opacity control to only make objects transparent instead of fully hiding / turning the mask transparent afterwards

# References

## Literature


[Ber+00] Marcelo Bertalmio et al. “Image inpainting”. In: *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH ’00. USA: ACM Press/Addison-Wesley Publishing Co., 2000, pp. 417–424. DOI: 10.1145/344779.344972 (cit. on p. 9).

[Che+22] Yi Fei Cheng et al. “Towards Understanding Diminished Reality”. In: *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. New York, NY, USA: ACM, 2022, pp. 1–16 (cit. on pp. 1, 5, 6).

[CJL23] Muzi Cui, Hao Jiang, and Chaozhuo Li. “Progressive-Augmented-Based DeepFill for High-Resolution Image Inpainting”. *Information (Basel)* 14.9 (2023), p. 512 (cit. on p. 2).

[DHZ15] Liang-Jian Deng, Ting-Zhu Huang, and Xi-Le Zhao. “Exemplar-Based Image Inpainting Using a Modified Priority Definition”. *PloS one* 10.10 (2015), e0141199 (cit. on p. 10).

[DRR19] Ding Ding, Sundaresh Ram, and Jeffrey J. Rodriguez. “Image Inpainting Using Nonlocal Texture Matching and Nonlinear Filtering”. *IEEE transactions on image processing* 28.4 (2019), pp. 1705–1719 (cit. on pp. 2, 9, 10).

[Elh+20] Omar Elharrouss et al. “Image Inpainting: A Review”. *Neural processing letters* 51.2 (2020), pp. 2007–2028 (cit. on p. 6).

[Gsa+24] Christina Gsaxner et al. “DeepDR: Deep Structure-Aware RGB-D Inpainting for Diminished Reality”. In: *Proceedings (International Conference on 3D Vision. Online) 3DV*. IEEE, 2024, pp. 750–760 (cit. on p. 2).

[Hon+24] Junlei Hong et al. “Visual Noise Cancellation: Exploring Visual Discomfort and Opportunities for Vision Augmentations”. *ACM transactions on computer-human interaction* 31.2 (2024), pp. 1–26 (cit. on p. 2).

[Liu+25] Weimin Liu et al. “Real-time tracking and inpainting network with joint learning iterative modules for AR-based DALK surgical navigation”. *Computer methods and programs in biomedicine* 272 (2025), p. 109068 (cit. on p. 6).

[MA23] Xiaojin Ma and Richard A. Abrams. “Visual Distraction’s “Silver Lining”: Distractor Suppression Boosts Attention to Competing Stimuli”. *Psychological science* 34.12 (2023), pp. 1336–1349 (cit. on p. 2).

[MF02] S. Mann and J. Fung. “EyeTap Devices for Augmented, Deliberately Diminished, or Otherwise Altered Visual Perception of Rigid Planar Patches of Real-World Scenes”. *Presence* 11 (Apr. 2002), pp. 158–175. DOI: 10.1162/1054746021470603 (cit. on pp. 1, 5).

[Mil+95] Paul Milgram et al. “Augmented reality: a class of displays on the reality-virtuality continuum”. In: *Telemanipulator and Telepresence Technologies*. Ed. by Hari Das. Vol. 2351. International Society for Optics and Photonics. SPIE, 1995, pp. 282–292. DOI: 10.1117/12.197321 (cit. on p. 1).

[MK94] Paul Milgram and Fumio Kishino. “A Taxonomy of Mixed Reality Visual Displays”. *IEICE Trans. Information Systems* vol. E77-D, no. 12 (Dec. 1994), pp. 1321–1329 (cit. on p. 5).

[Ram+16] Francois Rameau et al. “A Real-Time Augmented Reality System to See-Through Cars”. *IEEE transactions on visualization and computer graphics* 22.11 (2016), pp. 2395–2404 (cit. on p. 5).

[RLN07] Ruth Rosenholtz, Yuanzhen Li, and Lisa Nakano. “Measuring visual clutter”. *Journal of vision (Charlottesville, Va.)* 7.2 (2007), p. 17 (cit. on pp. 2, 4, 5).

[Set99] J. A. Sethian. “Fast Marching Methods”. *SIAM Review* 41.2 (1999), pp. 199–235. DOI: 10.1137/S0036144598347059. eprint: https://doi.org/10.1137/S0036144598347059 (cit. on p. 7).

[SHD21] Irawati Nurmala Sari, Emiko Horikawa, and Weiwei Du. “Interactive Image Inpainting of Large-Scale Missing Region”. *IEEE Access* 9 (2021), pp. 56430–56442 (cit. on pp. 2, 7).

[Sil17] Sanni Siltanen. “Diminished reality for augmented reality interior design”. *The Visual computer* 33.2 (2017), pp. 193–208 (cit. on p. 5).

[Tel04] Alexandru Telea. “An Image Inpainting Technique Based on the Fast Marching Method”. *Journal of Graphics Tools* 9 (Jan. 2004). DOI: 10.1080/10867651.2004.10487596 (cit. on pp. 7–9).

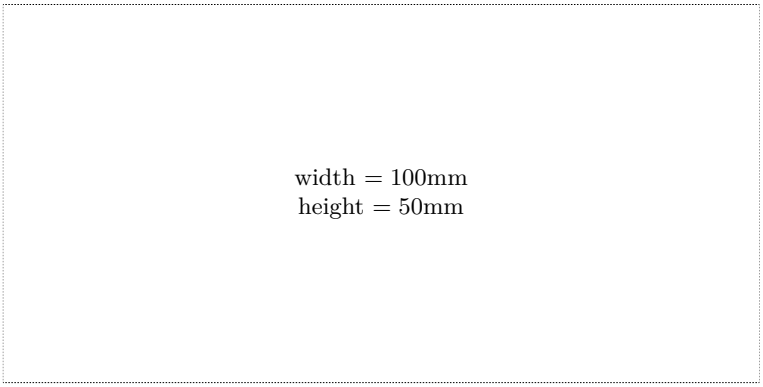
[Zha+22] Yunfan Zhang et al. “InDepth: Real-time Depth Inpainting for Mobile Augmented Reality”. *Proceedings of ACM on interactive, mobile, wearable and ubiquitous technologies* 6.1 (2022), pp. 1–25 (cit. on p. 6).

## Online sources

[Sta24] Statista. *Mobile augmented Reality (AR) users worldwide from 2023 to 2028*. 2024. URL: https://www.statista.com/statistics/1098630/global-mobile-augmented-reality-ar-users/ (visited on 08/31/2025) (cit. on p. 1).

# Check Final Print Size

— Check final print size! —




— Remove this page after printing! —